SystemVerilog类中的静态方法和属性

1.SystemVerilog类中的静态方法和属性

- 在SystemVerilog中关于方法有两种“static”
  - 一种是具有静态生命周期的方法
  - 另一种则是静态的方法，这种方法常见于类中

```
[task/function] static method_name(arg_list);   // static lifetime
static [task/function] method_name(arg_list);   // static method
```

- 在SystemVerilog中，类中方法的生命周期是动态的（automatic lifetime），**不能将类中的方法声明为具有静态的生命周期，但是可以将类中的方法声明为静态方法（static method）**

2.将类中方法声明为static lifetime

- 因为示例中在类packet中定义的方法disp被声明为了具有静态的生命周期，仿真器在对代码编译时产生warning提示在类中不能使用具有静态生命周期的方法
- 从示例中代码仿真结果可以看到，方法中的变量tmp在两次disp调用中，其值是保持每次调用的修改，第一次调用时tmp在方法结束时进行了自加一操作，第二次调用时，tmp保持了前一次disp调用时对于tmp的修改。这是因为disp被声明为了具有静态生命周期的方法，其中的变量也就具有静态的生命周期，即tmp具有静态的生命周期。**尽管可以进行仿真，并且有对应的仿真结果，但是这种在class中使用静态生命周期的方式在IEEE1800中是不允许的**

```
module top_tb;
class packet;
int svar;

task static disp(string name = "packet::");
    int tmp;
    $display("@%0t : static task start,%ssvar is %0h",$time,name,svar);
    $display("tmp is %0h",tmp);
    #10;
    $display("@%0t : static task end,%ssvar is %0h",$time,name,svar);
    tmp = tmp +1;
endtask
endclass

packet pkt;
initial begin
    pkt      = new();
    pkt.svar = 4;
    $display("================================");
    pkt.disp("pkt.");
    $display("================================");
    pkt.disp("pkt.");
    $display("================================");
end
endmodule
```

硅芯思见

```
Parsing design file 'top_tb.v'

Warning-[ISLSCM] Illegal static lifetime specified for class method
top_tb.v, 5
top_tb
  Task method inside class 'packet' cannot have static lifetime modifier
  preceding the method name.
  Please refer to Section 8.6 'Object methods' of the SystemVerilog
  1800-2012.
```

硅芯思见

```
================================
@0 : static task start,pkt.svar is 4
tmp is 0
@10 : static task end,pkt.svar is 4
================================
@10 : static task start,pkt.svar is 4
tmp is 1
@20 : static task end,pkt.svar is 4
================================
Simulation complete, time is 20.
```

硅芯思见

3.静态方法

- 不能将方法声明为具有静态的生命周期，但是可以将方法声明为静态方法

- 静态方法声明时需要在方法名前使用static关键字进行修饰，这与静态生命周期方法声明方式不同
- 类中的静态属性和静态方法可以直接通过类名的方式进行引用
- 类中的静态属性和静态方法也可以通过句柄名的方式进行引用
- **在SystemVerilog的类中，静态方法不能访问非静态属性**

```
module top_tb;
class packet;
static int svar;

static task disp(string name = "packet::");
    $display("@%0t : static task start,%ssvar is %0h",$time,name,svar);
    #10;
    $display("@%0t : static task end,%ssvar is %0h",$time,name,svar);
endtask
endclass

packet pkt1,pkt2;
initial begin
    packet::disp();
    packet::svar = 3; // static var assignment
    $display("================================");
    packet::disp();
    pkt1         = new();
    pkt2         = new();
    pkt1.svar    = 4;
    packet::disp();
    $display("================================");
    pkt1.disp("pkt1.");
    $display("================================");
    pkt1.disp("pkt2.");
    $display("================================");
    pkt1.svar    = 5;
    $display("================================");
    pkt1.disp("pkt1.");
    $display("================================");
    pkt1.disp("pkt2.");
    $display("================================");
end
endmodule
```

硅芯思见

```
@0 : static task start,packet::svar is 0
@10 : static task end,packet::svar is 0
====================================
@10 : static task start,packet::svar is 3
@20 : static task end,packet::svar is 3
@20 : static task start,packet::svar is 4
@30 : static task end,packet::svar is 4
====================================
@30 : static task start,pkt1.svar is 4
@40 : static task end,pkt1.svar is 4
====================================
@40 : static task start,pkt2.svar is 4
@50 : static task end,pkt2.svar is 4
====================================
====================================
@50 : static task start,pkt1.svar is 5
@60 : static task end,pkt1.svar is 5
====================================
@60 : static task start,pkt2.svar is 5
@70 : static task end,pkt2.svar is 5
====================================
Simulation complete, time is 70.
```

硅芯思见

4.静态属性

- 在SystemVerilog的类中，可以通过关键字static将属性声明为静态属性，通过静态属性可以在多个对象之间实现数据的共享，这里需要注意，静态属性不是某个对象所特有的。虽然静态属性可以实现数据在多个对象之间的共享，但是其并不会破坏类本身的隐藏特性，保证了类中属性的安全性。
- 因为静态属性对于所有对象来说都是共用的，其都只存储在一处，所以使用静态属性在某种程度上可以节省内存空间，同时也正是因为所有对象共享的原因，类的所有对象都能更新该值，每个对象对于静态属性的更新，其他所有对象都会获得更新后的值，通过这样的手段也可以提高数据属性的访问效率。

```
module top_tb;
class packet;
static int svar;

function new();
    svar = 8;
endfunction

endclass

packet pkt1,pkt2;
initial begin
    $display("@%0t : Beginning packet::svar is %0h",$time,packet::svar);
    pkt1 = new();
    $display("@%0t : After new() packet::svar is %0h",$time,pkt1.svar);
    $display("@%0t : After new() pkt1::svar is %0h",$time,pkt1.svar);
    fork
        begin : p1
            packet::svar = 5;
            $display("@%0t : packet::svar is %0h",$time,packet::svar);
            #10;
            packet::svar = 6;
            $display("@%0t : packet::svar is %0h",$time,packet::svar);
        end
        begin : p2
            #2 pkt1.svar = 7;
            $display("@%0t : pkt1.svar is %0h",$time,pkt1.svar);
            #10;
            $display("@%0t : pkt1.svar is %0h",$time,pkt1.svar);
        end
        begin : p3
            for (int loop = 0;loop <= 13;loop++) begin
                $display("@%0t : pkt2.svar is %0h",$time,pkt2.svar);
                #1;
            end
        end
    join
end
endmodule
```

硅芯思见



- **在静态方法中通过this引用类的属性时，此时不管该属性是否为静态的，在SystemVerilog中都是不允许的。**同时，**SystemVerilog在类的new函数调用时会自动调用super.new**，也可以在new函数中使用this，通过该示例我们也可以推断知道在SystemVerilog中new函数不能声明为static方法。

5.局部变量

Key Concepts

1. **Static Function**:
   - A static function operates independently of an object instance.
   - Variables declared **inside** the function are **local to the function**. They are not shared across different calls to the function unless explicitly declared as `static`.
2. **Local Variables in Static Functions**:
   - Variables like `pool` declared inside a static function are **local variables**. They are recreated each time the function is invoked and are **not inherently static**, even though the function itself is static.
3. **Static Variables**:
   - A truly static variable would persist across multiple calls to the function and would only be initialized once, retaining its value between calls.

Answering Your Question

Is `pool` a static variable?

No, `pool` is **not static** in this context. Even though the function is static, `pool` is a **local variable** that is re-declared and re-initialized every time the function executes.

Why can `pool` be assigned a new value?

Since `pool` is a local variable (non-static), it is newly created and initialized every time the function runs. The statement `pool = m_rsc[cntxt];` assigns a value to `pool`, and this assignment happens independently for each invocation of the function. There's no restriction on reassigning the value of `pool` because it is local.

Example to Illustrate

Here's a simplified analogy:

systemverilog 复制代码

```
class Example;
    static function void demo();
        int local_var;
        local_var = 10;  // Fine, local_var is local and initialized every call
        $display("local_var: %0d", local_var);
    endfunction
endclass
```

Even though `demo` is static, the `local_var` is not shared between calls. It is allocated and initialized every time `demo` is called.

Static Variables (True Static) Example

If `pool` were declared as `static` like this:

systemverilog 复制代码

```
class Example;
    static function void demo();
        static int local_static_var;
        local_static_var = 10;  // Can assign once, but retains value between ca
        $display("local_static_var: %0d", local_static_var);
    endfunction
endclass
```

In this case, the variable `local_static_var` would retain its value across calls, and you wouldn't be able to reassign it multiple times in the same way.

Conclusion

- The `pool` variable in your code is **local** to the function and is reinitialized every time the function is called.
- You can freely assign new values to `pool` for each function call.
- This behavior aligns with SystemVerilog's rules about local variables in static functions. Only explicitly declared `static` variables within the function would behave as persistent/static across calls.

6.静态变量初始化

SystemVerilog Example

systemverilog 复制代码

```
class Example;
    function void non_static_func();
        static int call_count = 0; // Static variable with an initial value of 0
        call_count++;              // Increment the static variable
        $display("Call count: %0d", call_count);
    endfunction
endclass

module testbench;
    Example example_instance;

    initial begin
        example_instance = new();

        // Call the non-static function multiple times
        example_instance.non_static_func(); // Call 1
        example_instance.non_static_func(); // Call 2
        example_instance.non_static_func(); // Call 3
    end
endmodule
```

Key Points in the Code

1. **Static Variable in a Non-Static Function**:
   - `call_count` is declared as `static` within the non-static function `non_static_func`.
   - Static variables are shared across all calls to the function and persist between calls.
2. **Initial Value**:
   - The static variable `call_count` is initialized to `0` only once when the simulation starts or the function is first executed.
   - It retains its value across multiple calls to the function.
3. **Non-Static Function**:
   - The function `non_static_func` itself is non-static, meaning it is tied to an instance of the class. Each instance can call the function, but the `static` variable is shared.

Expected Output

When you run the simulation, you'll see:

sql 复制代码

```
Call count: 1
```

```
Call count: 1
Call count: 2
Call count: 3
```

Analysis

1. **Static Variable Behavior**:
   - The `static` variable `call_count` retains its value across calls to `non_static_func`.
   - Even though the function is non-static, the static variable acts like global storage for the function.
2. **Initialization**:
   - The `call_count` variable is initialized to `0` only once.
   - On subsequent calls, it continues incrementing from its last value.
3. **Scope**:
   - `call_count` is not accessible outside the function, as it is scoped to the function. However, its lifetime persists across multiple function calls.
4. **Reusability**:
   - If you create another instance of `Example`, it will still share the same static variable across instances. This is because `static` variables are tied to the class, not to individual instances or function calls.

Conclusion

- Static variables within a non-static function can be useful for tracking shared data, such as the number of times a function is called, across multiple invocations.
- However, since the variable is shared, you should use it cautiously in a multi-threaded or instance-specific context to avoid unexpected behavior.

7.总结

- 静态属性在声明时需要使用关键字static;
- 静态属性被类的所有对象共享;
- 静态属性是静态存储的，即具有静态的生命周期;
- 静态属性引用的方式有两种：<类名>::<变量名>和<对象名>.<变量名>;
- 静态方法的生命周期是相对于类而言的，具有静态生命周期的方法指的是方法中的参数和变量的生命周期;
- 类中的方法和类中的属性默认都是automatic，类中的方法不能声明为静态生命周期的方法;
- 不能在静态方法中使用“this指针”和“super指针”